

ADR 0001 – User needs across bounded contexts

Document ID	adr-0001-bounded-context-user-needs
Revision	e422801
Date	2026-10-03
Status	Controlled in git

Contents

1 Context	3
2 Decision	3
3 Verification is anchored on design inputs (amendment)	4
3.1 Tests + Allure tags are the executable behaviour spec (“live BDD”)	5
4 V&V mapping	5
5 ID scheme	6
6 Traceability (many-to-many)	6
7 Gates (implemented)	6
8 Why “product need” is dropped	7
9 Consequences (implemented)	7
10 Litmus test	7

Vocabulary note (Design Review 25). Where this ADR says the test *is* the acceptance criterion, read: the design input is the acceptance criterion and its tagged test verifies it (the glossary, `CONTEXT.md`). The decision is unchanged.

Status: Proposed (design only; supersedes the earlier “product need” draft — that vocabulary is dropped).

Amended (Design Review 17): the per-context `satisfies` list below is gone. It restated what the design inputs already say — a context serves the needs its inputs (owned or realised) `traces_to` — and it drifted. Read `satisfies` in this record as that derived relation; RDM ignores the key.

1 Context

We model **one design document per bounded context** (DDD) — a single `kind: design document` per context that carries both its design inputs (the “what”) and its design output (the “how”). A user need frequently spans contexts (it is solution-independent and does not know about our context boundaries). Earlier drafts introduced a separate “product need” layer to hold cross-context journeys; that was redundant, because **validation is against the user need** (21 CFR 820.30(g): “devices conform to defined user needs and intended uses”). The journey *is* the user need. So we ditch “product need” and keep a single need vocabulary.

2 Decision

There is one need concept: the **user need**.

- A **user need** is the validated, solution-independent statement of what the user / intended use requires (the journey). It is the **validation** anchor.
- A user need is **cross-cutting: multiple SDDs may address the same user need** (many-to-many). It is therefore **not owned by any single SDD**.
- It is defined **once**, in the **validation & verification plan** — the document whose job is to plan validation, and validation is defined *against* user needs. Architecture stays pure design; needs sit at the validation/input altitude:

```
# documents/verification_and_validation_plan.md
---
id: VVP-001
user_needs:
  - {id: UN-001, text: "A clinician is promptly alerted to dangerous changes in a patient's vitals"}
  - {id: UN-002, text: "..."}
---
```

- Each bounded context is captured in **one design document** (`kind: design`) that declares the user needs its design contributes to (`satisfies`) and the design inputs it owns (see the amendment below):

```
# documents/design/alarms.md
---
id: SDS-ALRM-001
kind: design          # discovery keys on this, never on filename/folder
context: alarms
satisfies: [UN-001]
---
```

- Within each contributing context, a user need is refined into **design inputs** (verified by `@allure-` tagged tests) and, optionally, **backlog tasks** (plan).

3 Verification is anchored on design inputs (amendment)

Status of this section: Accepted. Refines the original decision.

Validation answers “right thing?” against the **user need**; verification answers “built right?” — and §820.30(f) defines verification as **design output meeting design input**, not output meeting user need. So verification is anchored one rung below the user need, on the **design input**:

- A **design input** is a verifiable requirement the design must satisfy. It is declared once, as structured data, **inside the design document of the context that owns it** (`design_inputs` frontmatter) — design input and design output live in the *same* per-context document (input is the “what”, the prose below is the “how”). Each input `traces_to` the user need(s) it refines:

```
# documents/design/alarms.md
---
id: SDS-ALRM-001
kind: design
context: alarms
satisfies: [UN-001]
design_inputs:          # inputs this context OWNS
  - {id: DI-1, text: "...latency requirement...", traces_to: [UN-001]}
  - {id: DI-2, text: "...acknowledgement...", traces_to: [UN-001]}
realises: [DI-7]        # OPTIONAL: a shared input owned
elsewhere               # elsewhere
---
# Design Inputs    (the "what")
# Design Outputs  (the "how")
```

A design input is **cross-cutting** like a user need: when more than one context realises it, exactly one context **owns** it (declares it in `design_inputs`) and the others reference it via `realises`

— the same declare-once + reference pattern as `satisfies`. The verification denominator is the **union** of `design_inputs` across all design documents. There is no separate design-input registry document.

- Each design input is **verified** by the automated test tagged `@allure.story("DI-...")`. The **design-input set is the release-gate denominator**; a user need is verified-through when every design input that `traces_to` it passes, rolled up across contexts.

3.1 Tests + Allure tags are the executable behaviour spec (“live BDD”)

We keep the *value* of BDD — traceability, an executable specification, and a living document — while dropping Gherkin’s parallel DSL and step-definition glue:

- **The test IS the acceptance criterion / scenario.** There is no `.feature` file and no step registry to keep in sync with the code.
- **Traceability** is the `@allure.story("DI-...")` tag (required) linking the test to the design input it verifies; an optional `@allure.label("output", "...")` records the design output (component / SDD) exercised.
- **The reviewed spec** is the registries (design inputs + user needs), approved in git.
- **The living document** is the Allure report (and the generated traceability matrix), produced from executed results.
- **It stays agile:** tests evolve with the code each iteration; there is no separate specification artifact to maintain in lockstep.
- An `allure.step("...")` narrative is **available but optional and free-form** — it is **not** mandated and **need not** be Given/When/Then. A test need not follow any prescribed structure.

4 V&V mapping

Question	Against	Evidence
Validation — “right thing?”	the user need (journey)	usability (IEC 62366), clinical, simulated/actual use — human + AI-persona formative, recorded as controlled docs, approved in git
Verification — “built right?”	the design input (§820.30(f))	<code>@allure.story("DI-...")</code> test results; the test is the acceptance criterion

A user need is **met** when it is **validated** *and* every design input that `traces_to` it — across every context — is **verified**.

5 ID scheme

- User need: `UN-NNN` — global, because it is cross-cutting. The **validation** anchor.
- Design input: `DI-n` — declared in the registry, `traces_to` user need(s). The **verification** anchor; tests tag it `@allure.story("DI-n")`.
- Design output: referenced by the optional `@allure.label("output", "...")`.

6 Traceability (many-to-many)

Relation	Cardinality	Source
user need → SDDs	$N \leftrightarrow M$	each SDD's <code>satisfies</code>
user need → design inputs	$1 \leftrightarrow N$	each design input's <code>traces_to</code>
design input → tests	$1 \leftrightarrow N$ (spanning contexts)	<code>@allure.story("DI-...")</code> tags
design input → output	$1 \leftrightarrow N$ (optional)	<code>@allure.label("output", "...")</code>
user need → validation	$1 \leftrightarrow 1+$	V&V plan + validation records

The same user need is **referenced** by many SDDs and refined by many design inputs, never **duplicated**: it is defined once in the registry.

7 Gates (implemented)

- **design-gate**: each design document present, complete, approved; each `satisfies` resolves to a real user need; each design input `traces_to` a real user need.
- **release-gate**: every **design input** is *verified* (its `@allure.story("DI-...")` test passes, aggregated across contexts), and every user need is addressed by at least one design input. *Validation* evidence (human summative review per the V&V plan's approach table, plus formative persona runs) is recorded and reviewed but is **not machine-gated** today — the gate enforces the verification denominator; making UN-keyed validation records a gate condition is a candidate future hardening, not an implemented check.

Whether a passing test actually *proves* its design input is judged by the independent reviewer of the pull request (git is the controlled record). An earlier faithfulness gate with hash-pinned per-input verdicts was retired in Design Review 4.

8 Why “product need” is dropped

- Validation is against user needs, so the cross-context journey *is* a user need — a separate “product need” was a redundant name for it.
- The former “context user need” was really an acceptance criterion / requirement (verified, not validated) — it folds into acceptance criteria.

9 Consequences (implemented)

- `record/sdd.py`: read the user-needs registry from the V&V plan frontmatter (`user_needs`); discover design documents by `kind: design` (never by filename/folder); union each document's `design_inputs` (and read `satisfies / realises`); reconcile coverage.
- Templates: V&V plan frontmatter `user_needs`; per-context design document frontmatter `kind: design + satisfies + design_inputs`. The architecture document holds design only — no need listing; there is no separate design-input document.
- Gates + traceability matrix: the release-gate denominator is the union of `design_inputs`; aggregate verification per design input and roll up to the user need it `traces_to`; add validation presence/approval as a separate, human-evidenced check.

10 Litmus test

Can two contexts contribute to one user need without duplicating it? Yes — the need is defined once in the registry; each SDD references it via `satisfies`, and the design inputs that `traces_to` it are verified by their tagged tests wherever those tests live.